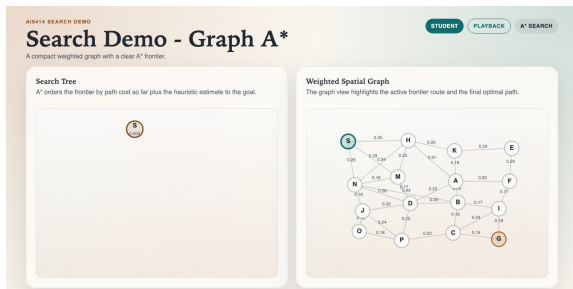


Frontiers in Graph Search

Stacks, queues, and priority queues on the small ai9414 graph

Same Graph, Different Frontier



Common idea: keep a frontier of discovered but not-yet-expanded paths.

The small graph starts at **S** and aims for **G**.

The first neighbours of S are:

H **M** **N**

What changes between algorithms is how the frontier chooses the next path.

DFS: Stack

Data structure: stack

Last in, first out.

To visit neighbours alphabetically, push them in reverse order:

H top: pop next

M

N bottom

Effect on the small graph

DFS commits to one branch before trying alternatives.

One DFS path is:

S -> H -> A -> B -> C -> G

This path has 5 edges. DFS does not guarantee a shallowest or lowest-cost path.

BFS: Queue

Data structure: queue

First in, first out.

After expanding S:

H front: dequeue next

M

N back

Effect on the small graph

BFS expands all depth-1 paths before depth-2 paths, and so on.

The BFS path is:

S -> N -> B -> C -> G

This path has 4 edges. In an unweighted graph, BFS finds a shallowest path.

UCS: Priority Queue by Cost So Far

Data structure: priority queue

Priority is accumulated path cost:

$$\text{priority} = g(n)$$

After expanding S:

$$\boxed{H} \quad g = 0.248$$

$$\boxed{N} \quad g = 0.258$$

$$\boxed{M} \quad g = 0.294$$

Effect on the small graph

UCS always expands the cheapest frontier path next.

The lowest-cost path is:

S → M → D → B → I → G

Total cost: 1.009.

With positive edge costs, UCS is lowest-cost optimal.

A*: Priority Queue by Cost Plus Heuristic

Data structure: priority queue

Priority combines cost so far and estimated cost to goal:

$$\text{priority} = f(n) = g(n) + h(n)$$

After expanding S:

$$\boxed{\text{M}} \quad f \approx 0.851$$

$$\boxed{\text{N}} \quad f \approx 0.936$$

$$\boxed{\text{H}} \quad f \approx 0.942$$

Effect on the small graph

A* chooses M first even though H has the cheapest single edge from S.

The returned path is:

S → M → D → B → I → G

Total cost: 1.009. With an admissible heuristic, A* is lowest-cost optimal.

Summary

Method	Frontier	Next path chosen by	Small-graph result
DFS	Stack	Most recently added path	S-H-A-B-C-G
BFS	Queue	Earliest added path	S-N-B-C-G
UCS	Priority queue	Lowest $g(n)$	S-M-D-B-I-G
A*	Priority queue	Lowest $g(n) + h(n)$	S-M-D-B-I-G

The graph can stay the same; changing the frontier changes the search behaviour.